Structured bindings for `std::extents`

Document #:	P2906R1 [Latest] [Status]
Date:	2026-05-10
Project:	Programming Language C++
Audience:	Library Evolution Working Group

Reply-to:	Bernhard Manfred Gruber <bernhardmgruber@gmail.com> Yihan Wang <yihwang@nvidia.com> Mark Hoemmen <mhoemmen@nvidia.com>

Contents

§ 1 Changelog

§ 1.1 R1

— Since [P1061R10] has been accepted, update the motivation of this proposal.

— Update the Godbolt implementation.

— Complete the proposal wording.

§ 1.2 R0

Initial revision

§ 2 Motivation and Scope

[P0009R18] proposed `std::mdspan` , which was approved for C++23. It comes with the utility class template `std::extents` to describe the integral extents of a multidimensional index space. Practically, `std::extents` models an array of integrals, where some of the values can be specified at compile-time. However, `std::extents` behaves very little like an array. A notable missing feature are structured bindings, which would come in handy if the extents of the individual dimensions need to be extracted:

Before

```
std::mdspan<double,
    std::extents<I, I1, I2,
        I3>, L, A> mdspan;
const auto& e =
        mdspan.extents();
for (I z = 0; z < e.extent(2);
        z++)
    for (I y = 0; y <
        e.extent(1); y++)
        for (I x = 0; x <
        e.extent(0); x++)
            mdspan[z, y, x] =
        42.0;

const auto total =
    e.extent(0) * e.extent(1) *
        e.extent(2);
```

After

```
std::mdspan<double,
    std::extents<I, I1, I2, I3>,
        L, A> mdspan;
const auto [depth, height, width]
        = mdspan.extents();
for (I z = 0; z < depth; z++)
    for (I y = 0; y < height; y++)
        for (I x = 0; x < width;
        x++)
            mdspan[z, y, x] =
        42.0;

const auto total =
    width * height * depth;
```

Comparing before and after, the usability gain with structured bindings alone is marginal, but it allows us to use descriptive names for the extents to improve readability.

The proposed feature becomes even more powerful in combination with [P1061R10] (adopted into C++26), which allows structured bindings to introduce a pack:

```
std::mdspan<double, std::extents<I, Is...>, L, A> mdspan;
const auto [...es] = mdspan.extents();
for     (const     auto     [...is]     :     std::views::cartesian_prod-
        uct(std::views::iota(0, es)...))
```

```
    mdspan[is...] = 42.0;

const auto total = (es * ...);
```

In this example, we trade readability for generality. Destructuring the extents into a pack allows us to expand the extents again into a series of `iota` views, which we can turn into the index space for `mdspan` using a `cartesian_product` . Notice that the implementation is rank agnostic, and for `std::layout_right` (`std::mdspan` 's default) iterates contiguously through memory. Under the proposed design (see Handling static extents), the pack `es` is heterogeneous: each element is either an `IndexType` or a `std::constant_wrapper` , so when every extent is static the fold `(es * ...)` itself yields a `std::constant_wrapper` and `total` is a compile-time constant.

§ 3 Impact On the Standard

This is a pure library extension. Destructuring `std::extents` in the current specification [N4944] is ill-formed, because `std::extents` stores its runtime extents in a private non-static data member, which is inaccessible to structured bindings. Some implementations nevertheless happen to accept structured bindings for `std::extents` due to representation details. Such bindings decompose the object representation, not the logical extents. This is especially misleading when any extent is static: static extents can disappear from the binding, and the remaining bindings no longer correspond to the multidimensional index space. For example, an implementation that stores only dynamic extents might let `std::extents<int, 4, std::dynamic_extent>{8}` decompose into one binding with value `8` , rather than two logical extents `4` and `8` . Providing a tuple interface gives users one portable decomposition over the logical extents and prevents code from depending on the wrong representation.

§ 4 Design Decisions

§ 4.1 Handling static extents

When destructuring `std::extents` we have to decide how to expose static extents to the user. This paper proposes to **retain the compile-time nature of static extents** by exposing them as `std::constant_wrapper` instances ([P2781R9]), while dynamic extents are exposed as plain values of `IndexType` .

This design has two important properties:

— It preserves information the user already gave to the compiler. Demoting static extents to runtime values would be lossy, with no way for user code to recover the compile-time nature afterwards.

— `std::constant_wrapper` has a non-explicit conversion to its `value_type` and overloads the usual arithmetic and comparison operators, so static extents transparently degrade to runtime values whenever used as such. As a result, the structured-binding

examples in Motivation and Scope work unchanged regardless of whether each extent is static or dynamic.

The cost is that destructured extents are heterogeneously typed: some bindings are values of `IndexType` , others are `constant_wrapper` specializations. In the rare cases where uniform typing is required, users can apply an explicit cast or use `std::extents::extent(I)` directly. The paper intentionally proposes preserving static extents. Always demoting static extents to runtime values is discussed in Alternative considered: demote static extents to runtime values only as a rejected alternative. LEWG should not choose that design unless it explicitly wants structured bindings to erase compile-time extent information; that is the outcome this proposal is designed to avoid.

§ 4.2 Modification of extents

Modifications of the values stored inside a `std::extents` should not be allowed, since it is neither possible in case of a static extent nor does it follow the design of `std::extents::extent(rank_type) -> index_type` , which returns by value.

§ 5 Proposed implementation

The proposed implementation uses the tuple interface and queries the extents type whether a specific extent is static or dynamic. Depending on this, `get` returns either the runtime extent via `std::extents::extent(I)` , or a `std::constant_wrapper` whose value is the static extent cast to `IndexType` :

```
namespace std {
    template <size_t I, typename IndexType, size_t... Extents>
    constexpr auto get(const extents<IndexType, Extents...>& e) noexcept {
         if constexpr (extents<IndexType, Extents...>::static_extent(I) ==
        dynamic_extent)
            return e.extent(I);
        else
            return constant_wrapper<
                static_cast<IndexType>(
                    extents<IndexType, Extents...>::static_extent(I))>{};
    }
}

template <typename IndexType, std::size_t... Extents>
struct std::tuple_size<std::extents<IndexType, Extents...>>
    : std::integral_constant<std::size_t, sizeof...(Extents)> {};

template <std::size_t I, typename IndexType, std::size_t... Extents>
struct std::tuple_element<I, std::extents<IndexType, Extents...>> {
    using type = decltype(std::get<I>(std::declval<std::extents<IndexType,
        Extents...>>()));
};
```

An example of such an implementation using GCC trunk's implementation of `<mdspan>` with `-std=c++26` on Godbolt is provided here: https://godbolt.org/z/sfMa5zEd5.

§ 6 Alternative considered: demote static extents to runtime values

A rejected alternative is to delegate `get` directly to `std::extents::extent(rank_type)`, which yields a uniform `IndexType` for every binding regardless of whether the corresponding extent is static or dynamic:

```
namespace std {
    template <size_t I, typename IndexType, size_t... Extents>
    constexpr IndexType get(const extents<IndexType, Extents...>& e) noex-
        cept {
        return e.extent(I);
    }
}

template <typename IndexType, std::size_t... Extents>
struct std::tuple_size<std::extents<IndexType, Extents...>>
    : std::integral_constant<std::size_t, sizeof...(Extents)> {};

template <std::size_t I, typename IndexType, std::size_t... Extents>
struct std::tuple_element<I, std::extents<IndexType, Extents...>> {
    using type = IndexType;
};
```

This alternative is simpler to specify, but that simplicity is bought by discarding the compile-time information that the user encoded in the static extents, with no way for user code to recover it. This paper does not recommend that design. It would make structured bindings look convenient while silently erasing exactly the static extent information that users chose `std::extents` to preserve.

§ 7 Wording

§ 7.1 Header <mdspan> synopsis 23.7.3.2 [mdspan.syn]

Modify the header <mdspan> synopsis in 23.7.3.2 [mdspan.syn] as follows:

```
namespace std {
  // ...

  // [mdspan.extents.dims], alias template dims
  template<size_t Rank, class IndexType = size_t>
    using dims = see below;

 // [mdspan.extents.tuple], tuple interface
 template<class T> struct tuple_size;

 template<size_t I, class T> struct tuple_element;

 template<class IndexType, size_t... Extents>
   struct tuple_size<extents<IndexType, Extents...>>;
```

```
  template<size_t I, class IndexType, size_t... Extents>
    struct tuple_element<I, extents<IndexType, Extents...>>;

  template<size_t I, class IndexType, size_t... Extents>
    constexpr tuple_element_t<I, extents<IndexType, Extents...>>
      get(const extents<IndexType, Extents...>& e) noexcept;

  // ...
}
```

§ 7.2 Tuple interface [mdspan.extents.tuple]

Insert a new subclause at the end of 23.7.3.3 [mdspan.extents], immediately after 23.7.3.3.7 [mdspan.extents.dims]:

23.7.3.3.8 Tuple interface [mdspan.extents.tuple]

```
template<class IndexType, size_t... Extents>
struct tuple_size<extents<IndexType, Extents...>>
  : integral_constant<size_t, sizeof...(Extents)> { };

template<size_t I, class IndexType, size_t... Extents>
struct tuple_element<I, extents<IndexType, Extents...>> {
  using type = conditional_t<
    extents<IndexType, Extents...>::static_extent(I) == dynamic_extent,
    IndexType,
    constant_wrapper<
      static_cast<IndexType>(
        extents<IndexType, Extents...>::static_extent(I))>>;
};
```

1 *Mandates*: `I < sizeof...(Extents)` is `true` .

```
template<size_t I, class IndexType, size_t... Extents>
constexpr tuple_element_t<I, extents<IndexType, Extents...>>
  get(const extents<IndexType, Extents...>& e) noexcept;
```

2 *Mandates*: `I < sizeof...(Extents)` is `true` .

3 *Returns*:

- `e.extent(I)` , if `extents<IndexType, Extents...>::static_extent(I) == dynamic_extent` ;
- otherwise, `constant_wrapper<static_cast<IndexType>(extents<IndexType, Extents...>::static_extent(I))>{}` .

§ 7.3 Feature-test macro

Modify the header `<version>` synopsis in 17.3.2 [version.syn] by adding the following macro definition, with the value selected by the editor to reflect the date of adoption of this paper:

```
+  #define  __cpp_lib_extents_structured_bindings  20XXXXL  //  also  in
       <mdspan>
```

§ 8 Acknowledgements

We would like to thank Christian Trott, Wenming Wang, and Jun Yang for reviewing and encouraging us to write this proposal.

§ References

[N4944] Thomas Köppe. 2023-03-22. Working Draft, Standard for Programming Language C++.

https://wg21.link/n4944

[P0009R18] Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. 2022-07-13. MDSPAN.

https://wg21.link/p0009r18

[P1061R10] Barry Revzin, Jonathan Wakely. 2024-11-24. Structured Bindings can introduce a Pack.

https://wg21.link/p1061r10

[P2781R9] Zach Laine, Matthias Kretz, Hana Dusíková. 2025-06-17. std::constexpr_wrapper.

https://wg21.link/p2781r9